

Developer Report

Test Environment

All findings were reproduced using Chrome DevTools on the Wayfair homepage.

Tools used

- Lighthouse
 - Performance
 - Coverage
 - Network
 - Layers
 - JavaScript-disabled testing
-

Finding 1 – Excessive JavaScript Execution

Mechanism

The homepage downloads and executes a large amount of JavaScript during the initial page load. Performance profiling shows that scripting dominates main-thread activity before the page becomes fully interactive. Coverage analysis also indicates that a substantial portion of the downloaded JavaScript is not used during the initial render.

Evidence / Reproduction

1. Open Chrome DevTools.
2. Open the **Performance** panel.
3. Record a page reload.
4. Inspect the Main Thread.

Observed:

- Scripting $\approx$ **5.2 s**
- Rendering $\approx$ **200 ms**

- Painting ≈ **226 ms**

Coverage panel:

- Approximately **50%** of downloaded resources were unused during the initial page load.

Recommended Fix

- Split large JavaScript bundles into smaller route-level chunks.
- Lazy-load below-the-fold components.
- Remove unused dependencies.
- Defer non-essential JavaScript until after the initial render.

Verification

Repeat the Performance and Coverage recordings.

Expected improvements:

- Reduced scripting time.
- Lower Total Blocking Time.
- Reduced unused JavaScript.
- Faster page interactivity.

Estimated Effort

Medium

Scope

Structural

Finding 2 – Third-Party JavaScript

Mechanism

Several third-party services execute JavaScript during page initialization. These scripts consume CPU time and compete with first-party application code during the critical rendering path.

Evidence / Reproduction

Record a Performance profile.

Execution is observed from services including:

- Google Tag Manager
- Stripe
- Quantum Metric
- Facebook
- Signifyd

Coverage also shows unused code associated with several third-party resources.

Recommended Fix

- Review all third-party dependencies.
- Remove integrations that are no longer required.
- Load analytics and marketing scripts after the primary content has rendered where possible.
- Delay optional services until user interaction.

Verification

Record another Performance profile.

Confirm:

- Reduced scripting time.
- Lower CPU usage from third-party scripts.
- Improved Total Blocking Time.

Estimated Effort

Low–Medium

Scope

Finding 3 – Large Initial Resource Download

Mechanism

The homepage loads a large number of CSS, JavaScript, and image resources during the initial page load. Coverage analysis indicates that some CSS and JavaScript are not required immediately.

Evidence / Reproduction

Open DevTools → Coverage.

Reload the page.

Observed:

- Significant unused CSS.
- Approximately half of downloaded resources unused during initial rendering.

Recommended Fix

- Remove unused CSS.
- Inline only critical CSS.
- Defer non-critical stylesheets.
- Continue optimising image delivery and asset loading.

Verification

Repeat Coverage analysis.

Expected improvements:

- Reduced unused CSS.
- Smaller transferred resources.
- Faster initial rendering.

Estimated Effort

Medium

Scope

Mostly Local

Finding 4 – Server Response Time

Mechanism

The initial HTML document requires approximately **804 ms** before the browser begins receiving the response. This delays all subsequent rendering work.

Evidence / Reproduction

Open DevTools → Network.

Reload the page.

Open the initial **Document** request.

Observed:

- Waiting for server response $\approx$ **804 ms**

Recommended Fix

Investigate:

- Backend processing time.
- Database queries.
- Cache configuration.
- CDN optimisation.
- Server-side rendering performance.

Verification

Repeat the Network recording.

Expected improvement:

- Reduced server response time.
- Lower Time to First Byte.

Estimated Effort

Medium–High

Scope

Structural

Finding 5 – Rendering Strategy

Mechanism

The homepage uses **Server-Side Rendering (SSR)** with client-side hydration. The initial HTML already contains meaningful page content before JavaScript executes.

Evidence / Reproduction

- Disable JavaScript in Chrome DevTools.
- Reload the homepage.

Observed:

- Navigation, hero banner, product listings, and layout remain visible.

View Source also contains rendered HTML rather than an empty application shell.

Recommended Action

No change recommended.

SSR is appropriate for an e-commerce homepage. Optimisation efforts should focus on reducing hydration cost rather than replacing the rendering strategy.

Verification

Not applicable.

Estimated Effort

None

Scope

Not Applicable